

Manual: Engenharia de Software

Processos e Ágil, Requisitos, UML 2.5, SOLID e GRASP, Padrões GoF, Arquiteturas e
Microsserviços (BFF, CQRS, Saga), REST e Segurança Web, Testes e SonarQube, Java/Spring,
Angular, Persistência, Git, CI/CD e Pontos de Função

CHANCELA EDITORIAL YLUNA85 LABS

Preparação Conjunta: Auditor Fiscal (TI) & Agente de Tributos Estaduais

Revisão Ortográfica e Glossário Geral de Siglas

Material de livre circulação interna. Revisado em Português Brasileiro (pt-BR).

1. Modelos de Ciclo de Vida e Processos de Desenvolvimento

A Engenharia de Software disciplina a aplicação de abordagens sistemáticas, estruturadas e quantificáveis ao desenvolvimento, operação e manutenção de software. O Ciclo de Vida compreende as fases pelas quais o sistema passa, desde a concepção inicial até o seu descarte.

- **Modelo Cascata (Waterfall / Clássico)**

Abordagem sequencial linear onde cada fase (Requisitos, Projeto, Codificação, Testes e Manutenção) deve ser 100% concluída antes de iniciar a próxima. Vantagem: alta previsibilidade e documentação rígida. Desvantagem: baixa flexibilidade a mudanças e entrega de software funcional apenas no final do ciclo.

- **Modelo Espiral (Barry Boehm)**

Modelo evolucionário e iterativo orientado à Análise e Gestão de Riscos. Cada ciclo da espiral é dividido em 4 quadrantes: 1. Determinação de Objetivos; 2. Avaliação e Redução de Riscos; 3. Desenvolvimento e Validação; 4. Planejamento da Próxima Fase.

- **Processo Unificado (RUP - Rational Unified Process)**

Processo iterativo e incremental centrado em arquitetura e guiado por Casos de Uso. Estruturado em 4 Fases temporais com marcos bem definidos: 1. Concepção (Inception - escopo e viabilidade do negócio); 2. Elaboração (Elaboration - arquitetura de referência e mitigação de riscos técnicos); 3. Construção (Construction - codificação e testes do produto); 4. Transição (Transition - homologação, implantação em produção e treinamento).

ALERTA DE PROVA - FASES DO RUP

A arquitetura de software é definida e a maior parte dos riscos técnicos é mitigada na fase de ELABORAÇÃO do RUP. Na fase de CONSTRUÇÃO ocorre o maior volume de codificação e geração de código!

2. Engenharia de Requisitos e Histórias de Usuário

A Engenharia de Requisitos compreende o conjunto de atividades voltadas para descobrir, analisar, documentar e manter os requisitos de um sistema computacional. Um requisito define o que o sistema deve fazer ou as restrições sob as quais ele deve operar.

- **Classificação dos Requisitos**

1. Requisitos Funcionais (RF): Descrevem as funções, serviços, comportamentos e transformações de dados que o software DEVE realizar (ex: 'O sistema deve calcular o ICMS devido'); 2. Requisitos Não Funcionais (RNF): Descrevem as restrições, propriedades emergentes e critérios de qualidade do sistema, classificados no modelo FURPS+ (Funcionalidade, Usabilidade, Confiabilidade/Reliability, Desempenho/Performance, Suportabilidade + Restrições de Design, Implementação, Interface e Físicas).

- **Processo de Engenharia de Requisitos**

Compreende 5 macroatividades interativas: 1. Elicitação / Levantamento (entrevistas, workshops, observação, prototipação); 2. Análise e Negociação (identificação de conflitos e inconsistências); 3. Especificação (documentação formal via SRS ou Casos de Uso); 4. Validação (revisões formais com clientes para garantir que o sistema atende às necessidades); 5. Gerenciamento de Requisitos (rastreamento e controle de mudanças).

- **Histórias de Usuário e o Critério INVEST**

Técnica ágil de especificação no formato: 'Como [persona], eu quero [ação], para que [benefício]'. Devem atender ao critério INVEST: Independent (independentes), Negotiable (negociáveis), Valuable (valiosas para o negócio), Estimable (estimáveis), Small (pequenas para caber em uma Sprint) e Testable (testáveis via critérios de aceite claros).

DIFERENÇA ENTRE RF E RNF NA PROVA

Requisito Funcional responde 'O QUE o sistema faz' (serviço ativo). Requisito Não Funcional responde 'COMO o sistema se comporta sob restrições de tempo, segurança, escalabilidade e plataforma'.

3. Modelagem de Sistemas com UML 2.5

A UML (Unified Modeling Language), mantida pelo OMG (Object Management Group), é a linguagem visual padrão da indústria para especificação, visualização, construção e documentação de artefatos de sistemas orientados a objetos. A UML (versão atual 2.5.1) divide seus 14 diagramas em Estruturais e Comportamentais.

- **Diagramas Estruturais (Aspectos Estáticos)**

1. Diagrama de Classes (classes, atributos, métodos, visibilidades [+ público, - privado, # protegido, ~ pacote] e relacionamentos: Associação, Agregação [todo-parte fraco], Composição [todo-parte forte com ciclo de vida dependente], Generalização/Herança e Realização/Interface); 2. Diagrama de Componentes (módulos e dependências de software); 3. Diagrama de Implantação / Deployment (nós de hardware, servidores e artefatos físicos de execução); 4. Diagrama de Objetos; 5. Diagrama de Pacotes. Também são estruturais o diagrama de estrutura composta e o diagrama de perfil, totalizando 7.

- **Diagramas Comportamentais (Aspectos Dinâmicos)**

1. Diagrama de Casos de Uso (atores, casos de uso e relacionamentos <<include>> [obrigatório/essencial] e <<extend>> [opcional/condicional sob ponto de extensão]); 2. Diagrama de Sequência (interação e troca de mensagens ordenadas no tempo entre linhas de vida); 3. Diagrama de Atividades (fluxo de controle e processamento paralelo com bifurcações/fork e junções/join); 4. Diagrama de Máquinas de Estados (transições de estados de um objeto disparadas por eventos). Também são comportamentais o diagrama de comunicação, o de visão geral de interação e o de tempo, totalizando 7. Nos casos de uso, a seta de <<include>> vai do caso base para o incluído, e a de <<extend>> vai do caso que estende para o caso base. Fragmentos combinados do diagrama de sequência (alt, opt, loop, par, ref) representam alternativas, repetições e paralelismo.

ALERTA DE PROVA - RELACIONAMENTOS NO CASO DE USO

O relacionamento <<include>> indica execução OBRIGATÓRIA e mandatária do caso de uso incluído. O relacionamento <<extend>> indica execução OPCIONAL e condicional dependente de um ponto de extensão!

4. Princípios de Projeto Orientado a Objetos e SOLID

O design de software de alta qualidade busca alcançar Alta Coesão e Baixo Acoplamento entre módulos. Os 5 princípios SOLID (formulados por Robert C. Martin - Uncle Bob) formam a base do desenvolvimento de software manutenível, extensível e testável.

- **Acoplamento versus Coesão**

Coesão: Grau em que as responsabilidades de uma classe ou módulo pertencem juntas (deve ser ALTA). Acoplamento: Grau de dependência e interconexão entre módulos diferentes (deve ser BAIXO/FRACO).

- **S - Single Responsibility Principle (Princípio da Responsabilidade Única)**

Uma classe deve ter um, e apenas um, motivo para mudar. Cada classe deve encapsular uma única responsabilidade funcional bem definida.

- **O - Open/Closed Principle (Princípio do Aberto/Fechado)**

Entidades de software (classes, módulos, funções) devem estar ABERTAS para extensão, mas FECHADAS para modificação. Novos comportamentos devem ser adicionados através de herança ou polimorfismo/interfaces, sem alterar o código-fonte original testado.

- **L - Liskov Substitution Principle (Princípio da Substituição de Liskov)**

Subtipos/classes derivadas devem ser capazes de substituir seus tipos base/superclasses sem que isso quebre a correteude e o comportamento esperado do programa.

- **I - Interface Segregation Principle (Princípio da Segregação de Interfaces)**

Muitas interfaces específicas para cada cliente são melhores do que uma única interface genérica e inflada. Nenhum cliente deve ser forçado a depender de métodos que não utiliza.

- **D - Dependency Inversion Principle (Princípio da Inversão de Dependência)**

Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de ABSTRAÇÕES. Abstrações não devem depender de detalhes; detalhes devem depender de abstrações. O DIP é o princípio; a injeção de dependências (DI) é uma técnica para aplicá-lo (as dependências são fornecidas de fora), e a inversão de controle (IoC) é o conceito mais amplo, presente em contêineres como o do Spring.

MNEMÔNICO DOS PRINCÍPIOS SOLID

SOLID: Single Responsibility (uma função), Open/Closed (estender sem alterar), Liskov (subclasses substituem a base), Interface Segregation (interfaces enxutas) e Dependency Inversion (depende de abstrações/interfaces)!

5. Padrões de Projeto GoF (Gang of Four)

Padrões de Projeto (Design Patterns) são soluções gerais, reutilizáveis e consagradas para problemas recorrentes no desenvolvimento de software orientado a objetos, catalogados por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides (GoF) em 23 padrões divididos em 3 categorias.

- Padrões Criacionais (Criação de Objetos)

1. Singleton (garante que uma classe tenha uma única instância global no sistema); 2. Factory Method (define uma interface para criar um objeto, deixando as subclasses decidirem qual classe instanciar); 3. Abstract Factory (fornece uma interface para criar famílias de objetos relacionados sem especificar suas classes concretas); 4. Builder (separa a construção de um objeto complexo da sua representação); 5. Prototype (cria novos objetos clonando uma instância existente).

- Padrões Estruturais (Composição de Classes e Objetos)

1. Adapter (converte a interface de uma classe em outra interface esperada pelos clientes, permitindo que classes incompatíveis trabalhem juntas); 2. Facade (fornece uma interface simplificada e unificada para um subsistema complexo); 3. Decorator (agrega dinamicamente novas responsabilidades e comportamentos a um objeto em tempo de execução sem usar herança); 4. Composite (compõe objetos em estruturas de árvore para representar hierarquias todo-parte); 5. Proxy (fornece um substituto ou marcador de localização para controlar o acesso a outro objeto).

- Padrões Comportamentais (Comunicação entre Objetos)

1. Observer (define uma dependência um-para-muitos entre objetos para que, quando um mude de estado, todos os dependentes sejam notificados automaticamente; é parecido com publicação e assinatura, mas no Observer o sujeito conhece seus observadores, enquanto no Pub/Sub há um intermediário que desacopla emissor e receptores); 2. Strategy (define uma família de algoritmos encapsulados, tornando-os intercambiáveis em tempo de execução); 3. Template Method (define o esqueleto de um algoritmo em uma superclasse, postergando etapas para as subclasses); 4. Command (encapsula uma requisição como um objeto); 5. Chain of Responsibility (passa requisições ao longo de uma cadeia de manipuladores).

DIFERENÇA ENTRE ADAPTER, FACADE E DECORATOR

Adapter CONVERTE interfaces incompatíveis. Facade SIMPLIFICA o acesso a um subsistema complexo. Decorator ADICIONA novos comportamentos dinamicamente sem alterar a interface original!

6. Arquitetura de Software: Monolitos, Microsserviços e Clean Architecture

A arquitetura de software estabelece a estrutura organizacional fundamental de alto nível do sistema, seus componentes, relacionamentos e restrições de comunicação.

- **Monolito versus Arquitetura de Microsserviços**

Monolito: Aplicação única e autocontida onde todas as camadas e módulos compartilham a mesma base de código e banco de dados centralizado. Microsserviços: Abordagem arquitetural que estrutura o sistema como uma coleção de serviços autônomos, fracamente acoplados, com implantação independente (independent deployment), escalabilidade granular e bancos de dados descentralizados por domínio (Database-per-Service).

- **Padrões Arquiteturais em Microsserviços**

API Gateway (ponto único de entrada que roteia requisições, faz balanceamento e autenticação); Service Discovery (registro e localização dinâmica de instâncias de serviços); Circuit Breaker (interrompe chamadas a serviços com falha contínua para evitar efeito cascata); Saga Pattern (gerencia transações distribuídas entre microsserviços via orquestração ou coreografia de eventos).

- **Clean Architecture e Arquitetura Hexagonal (Ports & Adapters)**

Organização em camadas concêntricas onde a Regra de Dependência determina que as dependências de código-fonte devem apontar SEMPRE PARA O CENTRO (em direção às regras de negócio de alto nível). Centro: Entidades (Domain Entities) e Casos de Uso (Use Cases), totalmente isolados e desacoplados de frameworks, bancos de dados e interfaces de usuário.

A REGRA DE OURO DA CLEAN ARCHITECTURE

O núcleo de negócio (Entidades e Casos de Uso) NUNCA deve conhecer ou depender do banco de dados, da interface gráfica ou de bibliotecas externas. Os detalhes de infraestrutura é que dependem das abstrações centrais!

7. Web Services e APIs RESTful

APIs RESTful baseiam-se no estilo arquitetural REST (Representational State Transfer), introduzido por Roy Fielding, normalmente implementado sobre HTTP, para comunicação interoperável entre sistemas distribuídos.

- **Os 6 Princípios Arquiteturais do REST**

1. Arquitetura Cliente-Servidor (separação de responsabilidades de UI e armazenamento); 2. Stateless (cada requisição do cliente deve conter TODAS as informações necessárias para ser processada, sem armazenar contexto de sessão no servidor); 3. Cacheable (as respostas devem ser explicitamente declaradas como passíveis ou não de cache); 4. Interface Uniforme (identificação de recursos por URIs, manipulação por representações JSON/XML, mensagens autoexplicativas e HATEOAS, isto é, hipermídia como motor do estado da aplicação); 5. Sistema em Camadas (a arquitetura pode ser composta por camadas intermediárias transparentes de proxy, gateway e segurança); 6. Código sob Demanda (opcional).

- **Métodos HTTP e Idempotência**

GET (recupera recurso; SEGURO e IDEMPOTENTE); POST (cria novo recurso; NÃO SEGURO e NÃO IDEMPOTENTE); PUT (substitui integralmente o recurso; NÃO SEGURO, mas IDEMPOTENTE); PATCH (atualização parcial do recurso; NÃO SEGURO e não necessariamente idempotente); DELETE (remove recurso; NÃO SEGURO, mas IDEMPOTENTE).

- **Códigos de Status HTTP Fundamentais**

1xx (Informativo); 2xx (Sucesso: 200 OK, 201 Created, 204 No Content); 3xx (Redirecionamento: 301 Moved Permanently, 304 Not Modified); 4xx (Erro do Cliente: 400 Bad Request, 401 Unauthorized [sem autenticação], 403 Forbidden [sem permissão], 404 Not Found); 5xx (Erro do Servidor: 500 Internal Server Error, 503 Service Unavailable).

CONCEITO DE IDEMPOTÊNCIA EM APIS

Uma operação HTTP é IDEMPOTENTE quando executá-la uma única vez ou múltiplas vezes consecutivas produz EXATAMENTE O MESMO resultado no servidor. São idempotentes: GET, PUT, DELETE e HEAD. O POST NÃO é idempotente!

8. Testes de Software e Garantia da Qualidade

O teste de software é o processo de execução de um programa com a intenção explícita de encontrar defeitos e verificar se o sistema atende aos requisitos especificados.

- **A Pirâmide de Testes**

Conceito de Mike Cohn, difundido por Martin Fowler. 1. Testes Unitários / Unidade (base da pirâmide: alta quantidade, execução em milissegundos, baixo custo, testam classes e métodos isolados com Mocks); 2. Testes de Integração (meio: testam a comunicação entre múltiplos módulos, APIs e banco de dados); 3. Testes Ponta a Ponta / E2E e UI (topo: menor quantidade, execução mais lenta, testam o fluxo completo do usuário no navegador).

- **Testes Caixa-Preta (Funcionais)**

Testam o software sem conhecimento do código-fonte interno. Técnicas: Partição de Equivalência (divide as entradas em classes válidas e inválidas) e Análise do Valor Limite (testa os valores nas bordas e fronteiras das classes de equivalência, onde os defeitos são mais frequentes).

- **Testes Caixa-Branca (Estruturais)**

Testam a lógica interna, fluxos e estruturas do código. Técnicas: Cobertura de Instruções/Linhas, Cobertura de Ramos/Decisões e Teste de Caminhos Básicos.

- **Complexidade Ciclomática de McCabe (V(G))**

Métrica que indica o número de caminhos linearmente independentes no grafo de fluxo de controle. $V(G) = E - N + 2P$, onde E é o número de arestas, N o de nós e P o de componentes conexos ($P = 1$ para um único programa). Equivale a $V(G) = \text{número de nós predcados (if, while, for, cada condição)} + 1$, quando as decisões são binárias. É também o número mínimo de casos de teste para cobrir todos os ramos independentes.

- **Test-Driven Development (TDD)**

Ciclo Red-Green-Refactor: 1. Escreve um teste que falha (Red); 2. Escreve o código mínimo necessário para fazer o teste passar (Green); 3. Refatora o código para eliminar duplicações e melhorar o design mantendo o teste passando (Refactor).

CÁLCULO RÁPIDO DA COMPLEXIDADE CICLOMÁTICA

Conte as decisões binárias (cada if, while, for e cada condição composta separada por AND ou OR) e some 1. Um trecho sem nenhuma decisão tem $V(G) = 1$.

9. DevOps, CI/CD e Controle de Versão com Git

A cultura DevOps integra desenvolvimento (Dev) e operações (Ops) por meio de automação contínua de entrega de software, testes automatizados e monitoramento de infraestrutura.

- **Controle de Versão Distribuído com Git**

Estrutura dos 3 Estados Locais: 1. Working Directory (arquivos em edição); 2. Staging Area / Index (arquivos preparados para commit via 'git add'); 3. Local Repository (histórico de commits salvos via 'git commit'). Comandos avançados: 'git rebase' (reescreve o histórico aplicando commits lineares), 'git merge' (cria commit de mesclagem), 'git cherry-pick' (aplica commit específico de outra branch) e 'git stash' (salva alterações temporárias em pilha sem comitar).

- **Pipelines de CI/CD (Integração e Entrega Contínua)**

Integração Contínua (CI): Desenvolvedores mesclam seu código frequentemente no repositório principal, disparando builds automáticos e suítes completas de testes unitários; Entrega Contínua (Continuous Delivery): O software aprovado no CI é empacotado e preparado automaticamente para implantação em produção mediante aprovação manual; Implantação Contínua (Continuous Deployment): Toda alteração aprovada no pipeline é implantada em PRODUÇÃO de forma 100% AUTOMÁTICA sem intervenção humana.

- **Estratégias Modernas de Deploy**

Blue-Green Deployment (mantém dois ambientes idênticos em produção, alternando o roteamento do roteador/balanceador); Canary Deployment (libera a nova versão para uma pequena porcentagem de usuários antes de liberar para 100% da base).

DIFERENÇA ENTRE CONTINUOUS DELIVERY E CONTINUOUS DEPLOYMENT

Continuous Delivery requer uma APROVAÇÃO MANUAL de um operador para subir em produção. Continuous Deployment sobe para produção de forma 100% AUTOMATIZADA assim que os testes passam!

10. Armadilhas Conceituais Recorrentes em Engenharia de Software

Pontos conceituais que costumam gerar erro em questões de Engenharia de Software. Servem de revisão rápida; a base de cada ponto está nas seções anteriores.

- **Pegadinha 1: POST NÃO é Idempotente**
GET, PUT e DELETE são idempotentes. Executar múltiplos POSTs cria múltiplos recursos duplicados no servidor.
- **Pegadinha 2: Singleton vs Factory Method**
Singleton garante instância única. Factory Method encapsula a lógica de instanciação deixando subclasses decidirem.
- **Pegadinha 3: Complexidade Ciclomática = Predicados + 1**
A complexidade ciclomática é o número de decisões binárias mais um, que também indica o número mínimo de casos de teste para cobrir os caminhos independentes.
- **Pegadinha 4: Clean Architecture Aponta para Dentro**
Todas as dependências de código apontam para o centro (regras de negócio). Entidades nunca conhecem o banco de dados.
- **Pegadinha 5: <<include>> é Obrigatório, <<extend>> é Opcional**
No Diagrama de Casos de Uso, include é mandatório e extend depende de condição específica.
- **Pegadinha 6: TDD é Red -> Green -> Refactor**
A ordem do TDD é: 1. Falha (Red); 2. Passa (Green); 3. Melhora (Refactor). Nunca se escreve o código antes do teste.
- **Pegadinha 7: Agregação vs Composição**
Composição é vínculo forte com destruição conjunta (ex: Pedido e Itens do Pedido). Agregação é vínculo fraco independente.
- **Pegadinha 8: Inversão de Dependência (DIP)**
Módulos de alto nível não dependem de baixo nível; ambos dependem de abstrações (interfaces).
- **Pegadinha 9: Rebase Lineariza o Histórico**
Git rebase reescreve o histórico para criar uma linha contínua, enquanto merge cria um commit de junção com dois pais.
- **Pegadinha 10: RUP Define Arquitetura na Elaboração**
A fase de Elaboração mitiga riscos e fecha a arquitetura de base; a Construção foca em codificação massiva.

SÍNTESE FINAL DE ENGENHARIA DE SOFTWARE

POST não é idempotente; include é obrigatório e extend, opcional; complexidade é predicados mais 1; a arquitetura limpa aponta para o centro; no TDD o teste nasce vermelho.

11. Outros Modelos de Processo e Agilidade

Além de cascata, espiral e RUP, há modelos incrementais, orientados a prototipação e os métodos ágeis.

- **Incremental e iterativo**

No incremental, o sistema é entregue em partes (incrementos), cada uma funcional. No iterativo, o trabalho é repetido em ciclos para refinar o produto. Muitos processos combinam os dois.

- **Prototipação**

Constrói uma versão preliminar para esclarecer requisitos. Protótipo descartável (throwaway) serve só para explorar e é jogado fora; protótipo evolutivo vira a base do produto final.

- **Modelo em V**

Relaciona cada fase de desenvolvimento a um nível de teste: requisitos a testes de aceitação, projeto de alto nível a testes de sistema, projeto detalhado a testes de integração e codificação a testes unitários. Enfatiza verificação e validação.

- **Manifesto Ágil**

Quatro valores: indivíduos e interações mais que processos e ferramentas; software funcionando mais que documentação abrangente; colaboração com o cliente mais que negociação de contratos; responder a mudanças mais que seguir um plano. Os itens à direita também têm valor, mas os da esquerda valem mais. São 12 princípios.

- **Extreme Programming (XP)**

Valores: comunicação, simplicidade, feedback, coragem e respeito. Práticas: programação em par, TDD, integração contínua, refatoração, propriedade coletiva do código, releases pequenos, jogo do planejamento e padrão de codificação.

- **ISO/IEC/IEEE 12207**

Norma de processos do ciclo de vida de software: define processos de aquisição, fornecimento, desenvolvimento, operação, manutenção e suporte, com base para modelos de maturidade como o MPS.BR.

INCREMENTO ENTREGA PARTES, ITERAÇÃO REFINA

Incremental: acrescenta funcionalidades a cada entrega. Iterativo: revisita e melhora o que já existe. Os métodos ágeis são iterativos e incrementais.

12. Requisitos em Profundidade: Elicitação, Priorização e Qualidade

Complementa a seção 2 com técnicas de levantamento, priorização, critérios de aceitação e o modelo de qualidade ISO/IEC 25010.

- **Técnicas de elicitación**

Entrevistas, questionários, observação e etnografia (útil para requisitos do trabalho real), oficinas e JAD (Joint Application Design), brainstorming, cenários e casos de uso, prototipação, análise de documentos e personas.

- **Priorização**

MoSCoW: Must (deve ter), Should (deveria ter), Could (poderia ter) e Won't (não desta vez). Modelo Kano: requisitos básicos, de desempenho (quanto mais, melhor), atrativos (encantadores) e indiferentes. Value Proposition Canvas: perfil do cliente (tarefas, dores e ganhos) e mapa de valor.

- **Hierarquia ágil**

Épico (grande demanda), feature (funcionalidade), história de usuário e tarefa. Mapeamento de histórias organiza o trabalho pela jornada do usuário. Definição de Pronto (DoD) vale para todo item; critérios de aceite valem para cada história.

- **Crítérios de aceite e BDD**

BDD (desenvolvimento guiado por comportamento) descreve exemplos no formato Gherkin: Dado (Given) um contexto, Quando (When) uma ação, Então (Then) um resultado. Serve de documentação e de teste automatizado.

- **Design Thinking e Lean Inception**

Design Thinking (etapas da d.school de Stanford): empatizar, definir, idear, prototipar e testar. Lean Inception (Paulo Caroli) é uma oficina curta para alinhar visão, personas, funcionalidades e o MVP (produto mínimo viável).

- **ISO/IEC 25010:2011**

Modelo de qualidade de produto com oito características: adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

- **Documento de requisitos**

A especificação de requisitos de software (SRS) é orientada pela ISO/IEC/IEEE 29148. Rastreabilidade liga requisitos a projeto, código e testes.

MOSCOW ORDENA ENTREGAS, KANO MEDE SATISFAÇÃO

MoSCoW classifica o que entra na entrega. O modelo Kano explica como cada tipo de requisito afeta a satisfação do cliente.

13. GRASP, Coesão, Acoplamento e Outros Princípios

Complementam o SOLID com padrões de atribuição de responsabilidades e critérios de qualidade do projeto.

- **GRASP (Larman): os nove padrões**

Especialista na Informação (a responsabilidade vai para quem tem a informação); Criador (quem cria instâncias); Controlador (recebe eventos da interface e coordena); Baixo Acoplamento; Alta Coesão; Polimorfismo; Fabricação Pura (classe artificial para preservar coesão); Indireção (intermediário para reduzir acoplamento); Variações Protegidas (proteger pontos de mudança por interfaces).

- **Tipos de coesão (da melhor à pior)**

Funcional, sequencial, comunicacional, procedural, temporal, lógica e coincidental.

- **Tipos de acoplamento (do menor ao maior)**

De dados, de carimbo (stamp), de controle, externo, comum (dados globais) e de conteúdo (um módulo acessa o interior de outro).

- **Outros princípios**

DRY (não repita a si mesmo), KISS (mantenha simples), YAGNI (você não vai precisar disso: não construa o que não é necessário) e Lei de Deméter (fale só com seus vizinhos imediatos). Preferir composição a herança.

- **Pilares da orientação a objetos**

Abstração, encapsulamento, herança e polimorfismo.

ESPECIALISTA NA INFORMAÇÃO, CRIADOR É OUTRO PADRÃO

Quem tem a informação recebe a responsabilidade de usá-la (Especialista). Quem cria instâncias de uma classe segue o padrão Criador. Não confundir.

14. Padrões GoF Complementares

Completam os 23 padrões do GoF: 5 criacionais, 7 estruturais e 11 comportamentais.

- **Estruturais restantes**

Bridge (separa abstração e implementação para variarem de forma independente) e Flyweight (compartilha objetos pequenos para economizar memória). Os cinco vistos na seção 5 (Adapter, Facade, Decorator, Composite, Proxy) completam os sete.

- **Comportamentais restantes**

Iterator (percorre uma coleção sem expor sua estrutura), Mediator (centraliza a comunicação entre objetos), Memento (guarda e restaura o estado de um objeto), State (o comportamento muda com o estado), Visitor (adiciona operações sem alterar as classes) e Interpreter (representa e interpreta uma gramática).

- **Singleton: cuidados**

Garante uma instância e um ponto global de acesso. Em ambientes com várias threads, exige inicialização segura. É criticado por criar estado global e dificultar testes.

- **Builder e Factory**

Builder constrói objetos complexos passo a passo (útil com muitos parâmetros opcionais). Factory Method delega a criação a subclasses; Abstract Factory cria famílias de objetos relacionados.

- **Proxy: variações**

Proxy virtual (adia a criação de objetos caros), remoto (representa um objeto em outro espaço de endereçamento) e de proteção (controla o acesso).

- **Padrões de arquitetura de integração**

Os padrões de integração corporativa (EIP: canais, roteamento e transformação) estão na apostila de Middleware e Mensageria.

DECORATOR ACRESCENTA, PROXY CONTROLA, ADAPTER CONVERTE

Os três envolvem outro objeto, mas com objetivos distintos: Decorator adiciona comportamento, Proxy controla o acesso e Adapter adapta a interface.

15. Estilos de Arquitetura: Camadas, MVC, SOA e Arquitetura Orientada a Eventos

Complementa a seção 6 com estilos clássicos e a comparação entre SOA e microsserviços.

- **Arquitetura em camadas**
Apresentação, aplicação (ou negócio), persistência e banco de dados. Cada camada usa a inferior. Facilita a manutenção, mas pode gerar dependências rígidas e sobrecarga.
- **MVC, MVP e MVVM**
MVC: Model (dados e regras), View (interface) e Controller (recebe entradas e coordena). MVP: o Presenter intermedia a View e o Model. MVVM: o ViewModel expõe dados e comandos à View, com vinculação de dados (data binding), usado em Angular e em aplicações móveis.
- **SOA versus microsserviços**
SOA (arquitetura orientada a serviços): serviços de negócio reutilizáveis integrados em nível corporativo, muitas vezes por um barramento (ESB) e por SOAP. Microsserviços: serviços menores, independentes e implantáveis separadamente, com dados próprios, comunicação leve (HTTP e mensagens) e lógica nas pontas ("endpoints inteligentes, tubos simples").
- **Arquitetura orientada a eventos**
Produtores emitem eventos e consumidores reagem, geralmente por um broker. Padrões descritos por Martin Fowler: notificação de evento, transferência de estado carregada pelo evento, event sourcing e CQRS.
- **Domain-Driven Design (DDD)**
Modelagem centrada no domínio (Eric Evans): linguagem ubíqua, contextos delimitados (bounded contexts), entidades, objetos de valor, agregados, repositórios e eventos de domínio. Os contextos delimitados orientam a divisão em microsserviços.
- **Atributos de qualidade**
Escalabilidade, disponibilidade, desempenho, segurança, modificabilidade e testabilidade. Decisões de arquitetura equilibram esses atributos, e um ganho em um pode custar outro.

MICROSSERVIÇO NÃO É SÓ "SERVIÇO PEQUENO"

O que caracteriza microsserviços é a independência de implantação e de dados por contexto de negócio, não apenas o tamanho.

16. Microsserviços: BFF, CQRS, Saga, Resiliência e Migração

Padrões usados para construir e operar sistemas de microsserviços, citados no programa de TI.

- **API Gateway e BFF**

API Gateway: ponto único de entrada (roteamento, autenticação, limites de uso). BFF (Backend for Frontend): um backend por tipo de cliente (web, mobile), que adapta as respostas às necessidades de cada interface.

- **CQRS**

Separa o modelo de comandos (escrita) do de consultas (leitura), que podem usar armazenamentos distintos, otimizados para cada função. Traz consistência eventual entre os dois lados.

- **Event Sourcing**

O estado é guardado como a sequência de eventos que o produziram, e não apenas o valor atual. Permite auditoria e reconstrução do estado por reaplicação; costuma andar junto com CQRS.

- **Saga**

Transação distribuída como sequência de transações locais, com ações compensatórias em caso de falha. Coreografia: cada serviço reage a eventos. Orquestração: um coordenador central dirige os passos.

- **Circuit Breaker e resiliência**

O circuit breaker tem três estados: fechado (chamadas passam), aberto (falha rápida, sem chamar o serviço) e semiaberto (testa se o serviço se recuperou). Outros padrões: timeout, retentativa com espera crescente e jitter, bulkhead (isolamento de recursos), limitação de taxa e fallback.

- **Service mesh**

Camada de infraestrutura (como Istio e Linkerd) com proxies sidecar ao lado de cada serviço, que cuidam de mTLS, roteamento, retentativas e telemetria sem alterar o código.

- **Strangler Fig e outbox**

Strangler Fig: migra um monolito aos poucos, desviando funcionalidades para novos serviços. Outbox: grava o evento na mesma transação do dado e o publica depois (veja a apostila de Middleware).

- **Doze fatores (12-factor app)**

Base de código única; dependências explícitas; configuração no ambiente; serviços de apoio como recursos anexos; separação entre build, release e run; processos sem estado; exposição por porta; concorrência por processos; descartabilidade; paridade entre desenvolvimento e produção; logs como fluxo de eventos; tarefas administrativas como processos pontuais.

SAGA COMPENSA, NÃO REVERTE

Como não há transação ACID única entre serviços, a saga desfaz efeitos com ações compensatórias (por exemplo, cancelar a reserva feita). O resultado é consistência eventual.

17. APIs e Segurança Web: CORS, CSRF, XSS, SSO e Certificados

A segurança de aplicações e APIs web envolve o navegador, o servidor e a identidade.

- **Política de mesma origem e CORS**

O navegador aplica a política de mesma origem (esquema, host e porta). O CORS (Cross-Origin Resource Sharing) permite que um servidor autorize origens diferentes por cabeçalhos como Access-Control-Allow-Origin. Requisições "não simples" fazem antes uma requisição prévia (preflight) com o método OPTIONS. CORS é uma proteção do navegador; não substitui autenticação nem autorização.

- **CSRF**

Falsificação de requisição entre sites: o navegador envia cookies automaticamente e um site malicioso induz o usuário autenticado a executar uma ação. Defesas: tokens anti-CSRF, cookies SameSite e verificação de origem. APIs que usam tokens no cabeçalho Authorization são menos expostas.

- **XSS**

Injeção de scripts que executam no navegador da vítima. Defesas: codificação de saída conforme o contexto, validação de entradas, Content Security Policy (CSP) e cookies HttpOnly (não acessíveis por script).

- **API Gateway, SSO e tokens**

O gateway centraliza autenticação, autorização e limites. SSO (login único) permite um só login para vários sistemas, por SAML ou OpenID Connect. Tokens JWT carregam declarações assinadas; validar assinatura, emissor, validade e público (veja a apostila de Segurança).

- **Certificados digitais**

TLS com certificados X.509 protege o canal. TLS mútuo (mTLS) autentica também o cliente, comum entre serviços. Certificados ICP-Brasil autenticam pessoas e organizações e assinam documentos.

- **JSON, XML, REST e SOAP**

JSON é leve e comum em REST. XML é verboso e tem esquema (XSD). SOAP é um protocolo baseado em XML com contrato WSDL e extensões (WS-Security). O modelo de maturidade de Richardson descreve os níveis de REST: 0 (um endpoint), 1 (recursos), 2 (verbos HTTP) e 3 (hipermídia).

- **Boas práticas de API**

Versionamento, paginação, respostas de erro padronizadas, códigos de status corretos (como 409 para conflito, 422 para entidade não processável, 429 para excesso de requisições), cache com ETag e Cache-Control, documentação com OpenAPI e chaves de idempotência para POST.

CORS PROTEGE O NAVEGADOR, NÃO A API

O CORS é imposto pelo navegador. Um cliente que não é navegador ignora CORS, então a API precisa de sua própria autenticação e autorização.

18. Testes Avançados, Qualidade de Código e Ferramentas

Complementa a seção 8 com tipos de teste, dublês, métricas de qualidade e as ferramentas citadas no programa.

- **Tipos de teste**
Regressão (garante que mudanças não quebraram o que funcionava), fumaça (smoke: verificação rápida do essencial), aceitação (validação pelo usuário), e não funcionais: carga, estresse, pico (spike) e resistência (soak), além de segurança e usabilidade.
- **Dublês de teste**
Dummy (preenche parâmetros), stub (respostas fixas), spy (registra chamadas), mock (verifica interações esperadas) e fake (implementação simplificada funcional, como um banco em memória).
- **Cobertura**
Instruções, ramos (decisões), condições e MC/DC (cada condição influencia de forma independente a decisão), exigida em software crítico. Cobertura alta não garante testes bons; teste de mutação avalia a qualidade dos testes.
- **Testes de segurança**
SAST (análise estática do código), DAST (teste dinâmico da aplicação em execução), IAST (instrumentação em tempo de execução) e SCA (análise de dependências de terceiros).
- **SonarQube**
Ferramenta de análise estática que reporta bugs, vulnerabilidades, pontos de atenção de segurança (security hotspots), code smells, duplicações e cobertura, com perfis de qualidade e portões de qualidade (quality gates) que bloqueiam a integração se as metas não forem atendidas.
- **JUnit e Mockito (Java)**
JUnit 5: anotações como `@Test`, `@BeforeEach` e `@AfterEach` e asserções. Mockito cria mocks e configura comportamentos com `when` e `thenReturn`, e verifica interações com `verify`.
- **Jest e Playwright**
Jest: testes em JavaScript e TypeScript com `describe`, `it` e `expect`, mocks e testes de snapshot. Playwright: automação de testes ponta a ponta em navegadores (Chromium, Firefox e WebKit), com espera automática.

MOCK VERIFICA INTERAÇÃO, STUB FORNECE RESPOSTA

Stub devolve dados prontos para o teste seguir. Mock também confere se as chamadas esperadas aconteceram.

19. Linguagens e Frameworks: Java e Spring Boot, Python, TypeScript, Angular e React Native

O programa cita linguagens e frameworks de uso corporativo. Aqui estão os conceitos centrais de cada um.

- **Paradigmas e tipagem**

Imperativo, procedural, orientado a objetos, funcional, declarativo e orientado a eventos. Tipagem estática (tipos verificados na compilação, como Java e TypeScript) ou dinâmica (verificados na execução, como Python); forte ou fraca conforme as conversões implícitas.

- **Java**

Compilado para bytecode e executado na JVM (com compilação JIT e coleta de lixo). Exceções verificadas (checked) e não verificadas. Recursos modernos: lambdas, streams e records. Contêineres e servidores de aplicação (veja a apostila de Middleware).

- **Spring Boot**

Simplifica o Spring com autoconfiguração, starters (dependências agrupadas), servidor embutido (Tomcat por padrão), configuração em application.properties ou YAML, perfis e o Actuator (métricas e saúde). O contêiner do Spring faz injeção de dependências. Spring Data simplifica o acesso a dados.

- **Python**

Linguagem dinâmica e interpretada, com tipagem forte. Na implementação CPython, o GIL limita a execução paralela de threads em código dependente de CPU (conferir a versão). Muito usada em automação, dados e IA.

- **TypeScript**

Superconjunto tipado do JavaScript, que é transpilado para JavaScript. Tipos estáticos, interfaces e genéricos ajudam a detectar erros antes da execução.

- **Angular**

Framework baseado em TypeScript para aplicações de página única (SPA). Componentes com templates, diretivas, serviços com injeção de dependências, roteamento, formulários e RxJS (observáveis) para fluxos assíncronos. Angular CLI gera e constrói projetos.

- **React Native**

Framework para aplicativos móveis multiplataforma com JavaScript ou TypeScript e React; renderiza componentes nativos de cada plataforma. Mantido pela Meta.

TIPAGEM ESTÁTICA DETECTA ANTES, DINÂMICA DETECTA AO EXECUTAR

Em Java e TypeScript, muitos erros de tipo aparecem na compilação. Em Python, aparecem em tempo de execução (a menos que se use verificadores de tipo à parte).

20. Persistência: Hibernate, Redis, MongoDB, Flyway e Liquibase

Tecnologias de persistência e de evolução de esquemas citadas no programa.

- **Hibernate e JPA**

JPA é a especificação de mapeamento objeto-relacional (ORM) do Jakarta EE; o Hibernate é sua implementação mais conhecida. Entidades mapeadas em tabelas; consultas em JPQL ou HQL. Carregamento preguiçoso (lazy) e ansioso (eager). Problema N+1: uma consulta principal mais N consultas adicionais, resolvido com junções de busca (fetch join) ou carga em lote. Cache de primeiro nível (sessão) e de segundo nível (opcional). Bloqueio otimista com campo de versão.

- **Redis**

Armazenamento chave-valor em memória, com estruturas como strings, hashes, listas, conjuntos, conjuntos ordenados e streams. Suporta expiração (TTL), publicação e assinatura, e persistência por snapshot (RDB) ou log (AOF). Replicação, Sentinel (failover) e Cluster, com 16.384 slots de hash. Porta padrão 6379. Usos: cache, sessões, filas e limitação de taxa.

- **MongoDB**

Banco de documentos (BSON). Conjuntos de réplicas (um primário e secundários, com eleição) e fragmentação (sharding) por chave de shard. Índices e pipeline de agregação. Porta padrão 27017.

- **Flyway**

Versiona migrações de banco: scripts SQL nomeados por versão (como V1__criar_tabela.sql) executados em ordem e registrados em uma tabela de histórico, com verificação de checksum para detectar alterações. Há migrações repetíveis (prefixo R__).

- **Liquibase**

Descreve mudanças em changelogs (XML, YAML, JSON ou SQL) formados por changesets, identificados por id, autor e arquivo e registrados em tabela de controle. Suporta reversão (rollback) e vários bancos.

- **Boas práticas**

Migrações versionadas e revisadas, mudanças compatíveis com versões anteriores (para implantar sem parada), e ambientes de homologação parecidos com a produção.

FLYWAY POR VERSÃO DE SCRIPT, LIQUIBASE POR CHANGESSET

Ambos versionam o esquema do banco. Flyway é centrado em scripts numerados; Liquibase, em changesets com id e autor, e suporta formatos declarativos.

21. DevOps no Desenvolvimento: Git Avançado, Gitflow, Jenkins, Tekton e SonarQube

Complementa a seção 9 com estratégias de ramificação e ferramentas de pipeline citadas no programa.

- **Fluxos de ramificação**

Gitflow: ramos main (produção) e develop (integração), mais feature, release e hotfix. GitHub Flow: main sempre implantável, com ramos curtos e pull requests. Trunk-based development: integração frequente em um tronco com ramos muito curtos e feature flags.

- **Comandos e cuidados**

git fetch baixa sem alterar; git pull é fetch mais merge (ou rebase). git reset move o ponteiro (soft, mixed, hard); git revert cria um commit que desfaz outro, seguro em histórico compartilhado. Regra de ouro: não reescreva (rebase) histórico já publicado. Fast-forward avança o ponteiro sem commit de mesclagem.

- **Jenkins**

Servidor de automação com controlador e agentes. O pipeline como código é descrito em um Jenkinsfile (sintaxe declarativa ou com script), com etapas (stages), passos (steps) e plugins.

- **Tekton**

Framework de CI/CD nativo do Kubernetes. Recursos: Task (passos executados em contêineres), Pipeline (encadeia tarefas), TaskRun e PipelineRun (execuções). Roda no cluster e é declarativo.

- **Portões de qualidade no pipeline**

Etapas típicas: compilar, testes unitários, análise estática (SonarQube com quality gate), varredura de dependências e de imagens, empacotamento (imagem Docker), implantação e testes ponta a ponta. Helm empacota a implantação no Kubernetes (veja Middleware).

- **Estratégias de liberação**

Blue-green, canário, implantação gradual (rolling) e feature flags, que ativam ou desativam funcionalidades sem nova implantação.

FEATURE FLAG SEPARA IMPLANTAÇÃO DE LIBERAÇÃO

Com feature flags, o código pode estar em produção desligado e ser ativado depois, reduzindo o risco. Isso é diferente de implantar uma nova versão.

22. Métricas de Software e Análise de Pontos de Função (APF)

A APF mede o tamanho funcional do software sob a ótica do usuário, independentemente da tecnologia. O edital SEFAZ-BA 2022 para a área de TI cita a Análise de Pontos de Função e o planejamento e controle de métricas de projeto.

- **Tipos de função (IFPUG)**

Funções de dados: ALI (arquivo lógico interno, mantido pela aplicação) e AIE (arquivo de interface externa, mantido por outra aplicação). Funções transacionais: EE (entrada externa, altera dados), SE (saída externa, com processamento) e CE (consulta externa, só recuperação).

- **Complexidade**

Cada função é classificada em baixa, média ou alta pela quantidade de tipos de dados (DER) e de tipos de registros (RLR) nos arquivos, ou de arquivos referenciados (ALR) nas transações.

- **Passos da contagem**

Definir o tipo de contagem, o escopo e a fronteira da aplicação; contar as funções de dados e as transacionais; obter os pontos de função não ajustados (PFNA); aplicar o fator de ajuste (opcional); obter os pontos de função ajustados (PFA).

- **Fator de ajuste (VAF)**

São 14 características gerais do sistema, cada uma com nota de 0 a 5. $VAF = 0,65 + 0,01 \times \text{soma das notas}$, variando de 0,65 a 1,35. $PFA = PFNA \times VAF$.

- **Outras métricas**

SNAP mede requisitos não funcionais. NESMA e COSMIC são outros métodos de medição de tamanho funcional. Pontos de história (story points) são estimativa relativa das equipes ágeis, e a velocidade é a quantidade entregue por iteração.

- **Uso**

Estimativa de esforço e prazo (produtividade em horas por ponto de função), contratação de fábrica de software por resultado, acompanhamento de escopo e benchmarking.

FUNÇÃO DE DADOS É ARQUIVO LÓGICO, TRANSACIONAL É FLUXO

ALI e AIE são dados. EE, SE e CE são transações. A diferença entre ALI e AIE é quem mantém o arquivo: a própria aplicação (ALI) ou outra (AIE).

23. Aprofundamento: Padrões de Microsserviços na Prática (Saga, CQRS e BFF)

A migração de monólitos para microsserviços introduz problemas que não existem em uma aplicação única (transações distribuídas, leitura versus escrita em escalas diferentes, múltiplos tipos de cliente), resolvidos por um conjunto específico de padrões arquiteturais cobrados em provas discursivas de arquitetura de software.

- **O Problema da Transação Distribuída**

Em um monólito, uma transação de banco de dados (commit ou rollback atômico) garante consistência entre todas as tabelas envolvidas. Em microsserviços, cada serviço tem seu próprio banco de dados, e não existe uma transação ACID que abranja vários serviços ao mesmo tempo; é preciso um padrão específico para manter a consistência entre eles.

- **Padrão Saga**

Sequência de transações locais, uma por serviço, coordenada por eventos ou por um orquestrador central. Se uma etapa falha, o Saga executa transações de compensação para desfazer o efeito das etapas já concluídas (por exemplo, estornar um pagamento já efetuado). Coreografia: cada serviço reage a eventos publicados pelos demais, sem um controlador central (mais desacoplado, porém mais difícil de rastrear). Orquestração: um serviço orquestrador central comanda explicitamente cada passo da saga (mais fácil de rastrear e testar, porém introduz um ponto de coordenação).

- **CQRS (Command Query Responsibility Segregation)**

Separa o modelo de escrita (Comandos, que alteram o estado) do modelo de leitura (Queries, que apenas consultam), podendo usar bases de dados e estruturas de dados diferentes e otimizadas para cada finalidade. Frequentemente combinado com Event Sourcing, em que o estado é reconstruído a partir do histórico completo de eventos, em vez de armazenar apenas o estado atual.

- **BFF (Backend for Frontend)**

Cria uma camada de backend dedicada para cada tipo de cliente (aplicativo móvel, aplicação web, parceiro externo), evitando que um único endpoint genérico tenha que atender a necessidades de dados muito diferentes entre esses clientes, e reduzindo o acoplamento entre o frontend e os detalhes internos dos microsserviços.

- **Circuit Breaker e Resiliência entre Serviços**

Padrão que monitora as chamadas a um serviço dependente e, após um número de falhas consecutivas, 'abre o circuito' e passa a rejeitar novas chamadas imediatamente (com uma resposta de fallback), evitando que uma falha em cascata em um serviço lento derrube o sistema inteiro. Após um intervalo, o circuito entra em estado 'semiaberto' para testar se o serviço já se recuperou.

ALERTA DE PROVA - SAGA NÃO É TRANSAÇÃO ACID

A Saga garante consistência EVENTUAL entre serviços, por meio de compensação, e não consistência IMEDIATA como uma transação ACID tradicional. Não confundir os dois conceitos em prova!

24. Referências e Conferência

Conferência realizada em 21/09/2026, com aprofundamento em 22/09/2026 (seção 23).

- **Referências**

Sommerville, Engenharia de Software; Pressman, Engenharia de Software; Gamma et al. (GoF), Padrões de Projeto; Robert C. Martin, Clean Code e Clean Architecture; Fowler (Patterns of Enterprise Application Architecture, e artigos sobre microsserviços, CQRS, Event Sourcing e Strangler Fig); Newman, Building Microservices; Evans, Domain-Driven Design; Larman, Applying UML and Patterns; Fielding (dissertação sobre REST) e RFC 9110; Manual de Práticas de Contagem do IFPUG; documentação de Spring, Angular, Hibernate, Redis, MongoDB, Flyway, Liquibase, Jenkins, Tekton, SonarQube, JUnit, Mockito, Jest e Playwright; Edital SEFAZ-CE 2026 (FCC), Anexo VI, e Edital SEFAZ-BA 2022 (FGV), Anexo I.

- **Corrigido nesta revisão**

Fórmulas em LaTeX (V(G)) reescritas; "Alto Acoplamento Coesivo" para Alta Coesão; DIP separado de injeção de dependências e IoC; Observer distinguido de Pub/Sub; Pirâmide de Testes atribuída a Mike Cohn e difundida por Fowler; "a maioria dos erros" nos valores limite suavizado; REST "sobre HTTP nativo e síncrono" reformulado e HATEOAS incluído; diagramas UML completados (7 + 7); seção de armadilhas atribuída às bancas sem evidência.

- **A conferir**

Detalhes de versões de Spring Boot, Angular e ferramentas; regras de contagem da APF na versão do manual do IFPUG adotada; comportamento do GIL na versão do Python usada.

REGRA DE CONTEÚDO

Este material não afirma nada sem fonte. Se houver divergência com a documentação oficial, ela prevalece.

GLOSSÁRIO DE SIGLAS E ABREVIATURAS

Consulte abaixo o significado e a expansão explicativa de todas as siglas contábeis, tributárias e de TI presentes nas apostilas de preparação:

ACID	Atomicidade, Consistência, Isolamento e Durabilidade. Propriedades fundamentais para transações em SGBDs.
BI	Business Intelligence. Inteligência de Negócios para mineração e análise estratégica de dados.
CAP	Consistência, Disponibilidade e Tolerância a Partições. Teorema limitador para sistemas distribuídos.
CEBRASPE	Centro de Seleção e de Promoção de Eventos. Banca examinadora oficial de concursos públicos.
CFC	Conselho Federal de Contabilidade. Órgão responsável pela edição das NBCs no território nacional.
CIAP	Controle de Crédito de ICMS do Ativo Permanente. Controle fiscal de apropriação de créditos de bens imobilizados.
CID	Confidencialidade, Integridade e Disponibilidade. Pilares de sustentação da Segurança da Informação.
COBIT	Control Objectives for Information and Related Technologies. Framework global de Governança de TI.
CPC	Comitê de Pronunciamentos Contábeis. Emissor de normas convergentes com padrões internacionais.
FGV	Fundação Getulio Vargas. Banca examinadora de certames e instituição de pesquisa econômica.
ICMS	Imposto sobre Operações Relativas à Circulação de Mercadorias e Prestações de Serviços de Transporte.
IPVA	Imposto sobre a Propriedade de Veículos Automotores. Tributo de competência estadual.
ITD	Imposto sobre Transmissão Causa Mortis e Doação. Tributo sobre heranças e doações na Bahia.
ITIL	Information Technology Infrastructure Library. Boas práticas de Gerenciamento de Serviços de TI.
KDD	Knowledge Discovery in Databases. Processo sistemático de descoberta de conhecimento em bases de dados.
LDO	Lei de Diretrizes Orçamentárias. Define as metas e prioridades para o orçamento do exercício seguinte.
LOA	Lei Orçamentária Anual. Estima as receitas e fixa as despesas públicas para o exercício financeiro.
NBC TA	Normas Brasileiras de Contabilidade Técnicas de Auditoria Independente. Alinhadas ao padrão internacional.
NBC TI	Normas Brasileiras de Contabilidade Técnicas de Auditoria Interna. Foco nos processos organizacionais.
NoSQL	Not Only SQL. Família de SGBDs não relacionais estruturados para alto desempenho e escalabilidade.
OLAP	Online Analytical Processing. Ferramentas analíticas baseadas em cubos de dados multidimensionais.
PAF-BA	Processo Administrativo Fiscal do Estado da Bahia. Lei nº 3.956/1981 que regula o contencioso.
PPA	Plano Plurianual. Planejamento estratégico governamental de médio prazo (4 anos).
RICMS-BA	Regulamento do ICMS do Estado da Bahia. Decreto regulamentador do principal imposto estadual.
RUP	Rational Unified Process. Processo de desenvolvimento de software disciplinado e preditivo.
SEFAZ-BA	Secretaria da Fazenda do Estado da Bahia. Órgão de arrecadação e controle fiscal baiano.

SGBD	Sistema de Gerenciamento de Banco de Dados. Software de controle e armazenamento seguro de dados.
SQL	Structured Query Language. Linguagem declarativa padrão para consulta em SGBDs relacionais.
TI	Tecnologia da Informação. Recursos de hardware, software e infraestrutura de rede corporativa.
XP	Extreme Programming. Metodologia ágil de engenharia de software focada em excelência técnica.